测试报告

1 黑盒测试报告

1.1 测试概述

本次黑盒测试面向 `RAG-Knowledge-Assistant` 项目展开，围绕用户管理、课程管理、课程资料管理、练习题管理、课程社区管理以及核心业务流程，对系统主要业务功能进行验证。

测试用例设计主要采用等价类划分、边界值分析、决策表法和场景法。测试执行采用“自动化接口测试为主、少量人工页面测试为辅”的方式，其中自动化测试承担主体覆盖任务，人工测试用于补充前端表单交互、PDF 预览、下载行为和页面级端到端流程验证，并结合实际运行结果与人工验证记录进行综合判断。

1.2 测试环境与工具

1.2.1 测试环境

1. 操作系统：Windows 11 开发环境。
2. 前端：React 19。
3. 后端：FastAPI。
4. 浏览器：Chrome 与 Edge。
5. 前端访问地址：`http://localhost:3000`。
6. 后端接口地址：`http://127.0.0.1:8000`。
7. 自动化测试环境：SQLite 测试库用于承载关系型业务数据；资料模块使用文档存储替身模拟文件行为。

1.2.2 测试工具

1. `Codex`：用于辅助生成测试计划、测试用例、自动化测试代码和测试报告整理内容。
2. `pytest + FastAPI TestClient`：用于执行自动化黑盒接口测试。
3. 浏览器人工操作：用于执行少量页面黑盒验证。

1.3 测试执行结果

1.3.1 自动化测试结果

自动化黑盒测试共执行 36 项，全部通过。

执行命令：

```
D:\miniconda3\envs\rag\python.exe -m pytest -q
```

执行结果：

```
36 passed, 9 warnings
```

自动化测试已覆盖用户管理、课程管理、资料管理、练习题管理、课程社区管理以及核心流程的主要黑盒接口验证，具体包括注册、登录、个人信息查看与修改、课程创建、加入与退出、资料上传、查看与删除、练习题查看与作答、社区发帖、评论、回复等主要功能点。

1.3.2 人工测试结果

人工黑盒测试共执行 10 项，全部通过。

人工测试主要补充验证了以下内容：

1. 注册页确认密码校验与角色选择校验
2. 登录后首页跳转、身份状态展示与刷新后的登录状态保持
3. PDF 资料预览效果与页面展示情况
4. 资料下载时的浏览器下载行为
5. 练习题页面反馈展示与“重新作答”后的状态恢复
6. 匿名发帖与匿名评论的展示效果
7. 回复评论后的页面定位与回显
8. 页面级核心流程端到端验证

1.3.3 综合统计

1. 自动化测试：36 项，通过 36 项。
2. 人工测试：10 项，通过 10 项。
3. 黑盒测试总执行数：46 项。
4. 黑盒测试总通过数：46 项。

自动化测试与人工测试结果一致，计划范围内的主要黑盒功能均完成验证。

1.4 缺陷与问题分析

本次黑盒测试未发现阻塞主要业务流程的严重缺陷，系统主要功能能够按需求正常运行，未出现导致系统无法使用、核心流程中断或角色权限错误的问题。

测试过程中记录到的轻微问题或风险如下：

1. 回复评论定位功能在页面滚动距离较长时，定位体验可能不够稳定。该问题未导致功能失效，因此未判定为失败项，但建议在后续版本中优化页面滚动与定位交互体验。
2. 自动化测试执行中出现若干技术栈弃用警告。这些警告主要与 Pydantic、SQLAlchemy 和 FastAPI 的版本兼容提示有关，不影响本次黑盒测试通过结果，但说明项目后续存在升级维护需求。

1.5 测试结论

本次黑盒测试结果表明，`RAG-Knowledge-Assistant` 项目在当前测试范围内整体表现良好，主要业务功能满足需求说明文档要求，教师与学生角色权限控制正确，课程平台核心业务流程能够顺利完成。

综合自动化测试与人工测试结果，可以认为该系统在黑盒测试范围内达到了课程作业要求，具备较好的功能完整性和可用性。

2 白盒测试报告

2.1 测试概述

本次白盒测试重点覆盖用户与认证、课程管理、课程资料管理三类业务。测试目标是基于代码结构验证关键逻辑的正确性与稳健性，并满足课程作业对白盒测试方法与

覆盖率指标的要求。

白盒测试的执行策略：优先通过后端接口触发真实业务调用链，贯通 `controller -> service -> repository -> dao`，仅对外部系统依赖（如文档存储客户端）使用替身隔离，以保证测试结果既具可重复性又具工程真实性。

2.2 测试环境与工具

2.2.1 测试环境

1. 运行环境：Python 3.11。
2. 后端框架：FastAPI。
3. 关系型数据测试环境：SQLite 内存数据库（测试会话）。
4. 文档存储测试环境：Mongo/GridFS 外部依赖替身。

2.2.2 测试工具

1. `Codex`：辅助设计测试用例、分析未覆盖路径和整理报告。
2. `pytest`：执行自动化白盒测试。
3. `FastAPI TestClient`：通过接口请求驱动后端完整调用链。
4. `pytest-cov`：统计覆盖率并输出未命中行。

2.3 测试对象说明

白盒测试覆盖如下代码模块：

1. 用户与认证调用链：`LoginController` → `LoginService` → `UserRepository` + `UserAuthRepository`；`UserController` → `UserService` → `UserRepository` + `UserAuthRepository`。
2. 课程管理调用链：`CourseController` → `CourseService` → `CourseRepository` + `UserRepository`。
3. 资料管理调用链：`rag_controller` → `DocumentDAO` → `document_repository`。

2.4 测试方法说明

2.5 测试执行结果

2.5.1 执行命令与通过情况

执行命令：`pytest tests/ -q`

测试结果：16 passed

2.5.2 覆盖率结果（本次执行）

```
pytest tests/ -q --cov=controller --cov=service \
--cov=repository --cov=dao --cov-report=term-missing
```

覆盖结果如下：

代码文件名	覆盖率	未覆盖行数
controller/CourseController.py	91%	18-22, 99, 114, 143
controller/ExerciseController.py	0%	1-70
controller/LoginController.py	85%	12-16
controller/PostController.py	0%	1-140
controller/UserController.py	100%	–
controller/__init__.py	100%	–
controller/rag_controller.py	100%	–
dao/__init__.py	100%	–
dao/document_dao.py	100%	–
repository/CommentRepository.py	0%	1-23
repository/CourseRepository.py	98%	34
repository/ExerciseRepository.py	0%	1-73
repository/PostRepository.py	0%	1-36
repository/UserAuthRepository.py	100%	–
repository/UserRepository.py	92%	19, 29
repository/__init__.py	100%	–
repository/document_repository.py	97%	38
service/CourseService.py	100%	–
service/ExerciseService.py	0%	1-74
service/LoginService.py	100%	–
service/PostService.py	0%	1-23
service/UserService.py	100%	–
service/__init__.py	100%	–
TOTAL	66%	共236行

2.6 未覆盖代码分析

未覆盖的语句并非测试遗漏或核心业务逻辑缺失，具体可分为以下三类：

- **防御性分支不可触发：**当前调用链中其它层会提前进行校验，提前返回或抛出异常，导致下游用于兜底处理的部分分支不可达。例如 service 层直接抛出 `HTTPException`，该异常由 FastAPI 框架统一捕获并转换为响应，controller 层中针对返回值为 `None` 的防御分支因此无法执行。

- **数据库生成器语句未覆盖：**`LoginController` 与 `CourseController` 中 `get_db()` 生成器的 `SessionLocal()` 与 `db.close()` 未被覆盖，原因是测试为保证可重复性

采用 SQLite 测试会话并通过依赖覆盖替换了默认数据库依赖，从而未执行生产环境函数体。

- **文档存储仓储层的生命周期函数未覆盖**：仅在应用启动/关闭事件中调用，不属于常规接口请求路径。

2.7 测试结果与分析

所有编写的测试用例均已通过，核心业务逻辑功能正常，无功能性缺陷或回归问题。

整体业务链路覆盖率??，主要业务分支与常见异常路径均经过验证，测试质量达标。

对于仍未命中的少量语句，已通过代码与调用链分析确认了其主要来源并进行了分析。